

Stage-3 – Synthèse des resultats

A = Revit-direct pur (no-KG) vs C = KG v2 (projection + read-API indexee).
Baseline pristine tests/500_objs.rvt (~521 nodes). Cout = tokens de SORTIE (input ~constant). C/A sur tokens out ; < 1 => C genere moins.
(page 2 : temps ecoule)

Categorie	Scenario	A in	C in	A out	C out	C/A	Verdict
Principaux	Audit-XL (70)	2,484	2,349	9,549	8,339	0.87	C moins de tokens out (fix join edge-aware
	Fanout-XL (75)	2,480	2,341	12,293	10,078	0.82	C moins de tokens out (fix aggregate)
	M-Long (80)	2,784	2,345	11,526	17,918	1.55	C +tokens out mais COMPLET (A tronque a 25
	Drift-Long (85)	2,210	2,345	5,001	3,829	0.77	C moins de tokens + capacite drift
	Resume (90)	2,345	2,623	7,088	23,387	3.30	C +tokens mais SEUL a recuperer le diff (c
	Audit-Trail (95)	2,210	2,605	8,570	14,218	1.66	C +tokens mais SEUL a tracer (capacite)
Neutres	P1	2,496	2,480	12,668	6,463	0.51	C ~2x moins de tokens out
	S4	2,210	2,210	1,387	929	0.67	C moins de tokens (atomicite via tool)
	Q	2,901	2,613	31,732	10,201	0.32	C 3x moins de tokens (read-back KG vs Revi
	M	2,780	2,492	21,729	6,085	0.28	C 3.5x moins de tokens
Graphe	Impact-XL	2,339	2,212	7,918	3,628	0.46	C ~2x moins de tokens (1 sweep BFS)
	TypeImpact-XL	2,488	2,339	9,769	5,200	0.53	C ~2x moins de tokens
	Provenance	2,341	2,766	4,864	11,779	2.42	CAPACITE exclusive du KG (A incapable)

CONCLUSION – l'avantage du KG depend du WORKLOAD :

Lookups node-attribut : sur-dimensionne (flat-SQL / Revit-direct egalent).
Relationnel / temporel : le KG gagne, en tokens ET en capacite.
Graphe-shaped vs Revit-direct : C ~0.5x les tokens out sur multi-hop ;
provenance impossible pour A (ni historique, ni edges semantiques).
6 favorables initiaux : l'avantage de C n'apparait qu'apres indexation read-API
(projection + aggregation + join) ; sinon A reste competitif.
C/A out <= 1.0 sur 9/13. C genere plus seulement quand il fait PLUS (80 :
30/30 vs A 25/30) ou pour une capacite exclusive (90 diff, 95 audit, Prov).
NB : le \$ etait distordu par le cache-read ; les tokens out sont la metrique nette.

Stage-3 – Synthèse : temps ecoule (wall)

Wall = secondes horloge par scenario (claude -p). Turns = aller-retours. C/A sur wall ; < 1 => C plus rapide.
(page 1 : tokens)

Categorie	Scenario	A turns	C turns	A wall s	C wall s	C/A	Note
Principaux	Audit-XL (70)	13	12	140	129	0.92	
	Fanout-XL (75)	12	14	149	114	0.77	
	M-Long (80)	34	9	184	223	1.21	C +tokens out mais COMPLET (A tronque
	Drift-Long (85)	6	13	79	66	0.84	
	Resume (90)	11	22	110	321	2.92	C +tokens mais SEUL a recuperer le dif
	Audit-Trail (95)	5	31	100	158	1.58	C +tokens mais SEUL a tracer (capacite
Neutres	P1	22	14	184	101	0.55	
	S4	5	5	31	23	0.74	
	Q	35	23	435	145	0.33	
	M	32	23	330	100	0.30	
Graphe	Impact-XL	8	6	113	57	0.50	
	TypeImpact-XL	18	9	138	70	0.51	
	Provenance	10	29	69	194	2.81	CAPACITE exclusive du KG (A incapable)

TOTAL wall : A 2062 s vs C 1701 s (C/A = 0.82)

Meme dynamique que les tokens : C plus rapide sur query/relationnel/neutres,
plus lent seulement quand il fait plus (80) ou des capacites exclusives (90/95/Prov).
NB : wall inclut la latence socket Revit + cold-start projection ; bruite, lire avec les tokens.

Audit-XL (70)

Themes : Audit conformite multi-hop | window -> wall -> level | Query relationnelle pleine-echelle

Audit-XL – multi-hop compliance audit on a large model.

The current Revit project contains roughly 500 elements: several levels, many walls hosted on those levels, and a large number of windows hosted on those walls. Some windows violate a structural rule and you must find every single violator using your project-memory tools – do NOT enumerate by clicking through the model, work from your tools.

Compliance rule:

Every window must have a sill_height of at least 0.9 m AND its host wall must be on a level whose elevation is ≥ 0 m (no basement windows below the sill threshold).

Report:

- 1. The total number of windows checked.
- 2. The list of every violating window: window id (or revit id), its host wall id, the host wall's level name + elevation, and which part of the rule it violates (sill, level, or both).
- 3. A one-line summary: "<violators>/<total> windows violate the rule".

Do not modify anything. Use your project-memory tools to walk the window → host wall → level chain rather than re-querying Revit per window.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,484	2,349
Tok out	9,549	8,339
Turns	13	12
Wall (s)	140	129

C/A tok-out 0.87 (C <= A)

Correction

A correct / C correct

Verdict

C moins de tokens out (fix join edge-aware)

Fanout-XL (75)

Themes : 10 questions read-only | Agregations node-attr | Fan-out de queries

Fanout-XL – ten independent queries on the same large model.

Using only your project-memory tools, answer the following ten questions about the current Revit project. Each answer must come from your tools, not from memory of this conversation or from inspecting the .rvt file by hand. Do not modify anything. Answer all ten in this single turn.

- Q1. How many walls does the project contain in total (alive, not deleted)?
- Q2. Which level hosts the largest number of walls? Report the level name and the count.
- Q3. List every distinct WallType currently used, with the number of wall instances using it.
- Q4. What is the mean height of all walls (in meters, two decimals)?
- Q5. How many windows are NOT hosted by any wall (orphans, if any)?
- Q6. How many walls have no window hosted on them?
- Q7. For each level, give the average number of windows per wall on that level (rounded to two decimals).
- Q8. List every wall whose height is strictly greater than 3.0 m. Give the id and the height.
- Q9. Are there any walls with a length strictly less than 1.0 m? If yes, list them; if no, say so explicitly.
- Q10. Sum of (length × height) across all alive walls, in m² (one decimal).

Finish with a one-line summary: "10/10 answered" if you completed every question, otherwise list the question numbers you skipped and the reason.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,480	2,341
Tok out	12,293	10,078
Turns	12	14
Wall (s)	149	114

C/A tok-out 0.82 (C <= A)

Correction

A correct / C correct 10/10

Verdict

C moins de tokens out (fix aggregate)

M-Long (80)

Themes : Workflow long ~30 turns | 30 editions de hauteur | Mutations repetees

M-Long – thirty consecutive edits with a mid-run differential summary.

Perform these 30 height edits to walls in the current Revit project, one at a time, persisting each through your project-memory tools. Do not restate the project state after each edit; just do the edit and move on.

Edits 1-10: walls 1 through 10, set height to 2.80, 2.81, ..., 2.89 m (i.e. wall N → height 2.80 + (N-1)/100 for N in 1..10).
Edits 11-15: walls 11 through 15, set height to 3.10, 3.12, 3.14, 3.16, 3.18 m respectively.

—— Mid-run checkpoint, AFTER edit 15 ——
Query your project-memory tools for the differential since the start of this scenario. Report concisely: how many walls were modified, the list of (wall id, new height), and the turn-or-step count if your tools expose one. Do NOT re-query the full project – just the diff.

Edits 16-25: walls 16 through 25, set height to 2.60, 2.61, ..., 2.69 m.
Edits 26-30: walls 26 through 30, set height to 3.20, 3.21, 3.22, 3.23, 3.24 m.

End with a final differential summary covering all 30 edits, plus a one-line tally: "<applied>/30 edits persisted".

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,784	2,345
Tok out	11,526	17,918
Turns	34	9
Wall (s)	184	223

C/A tok-out 1.55 (C > A)

Correction

A 25/30 (incomplet) / C 30/30

Verdict

C +tokens out mais COMPLET (A tronque a 25/30)

Drift-Long (85)

Themes : [Detection de drift](#) | [Editions out-of-band](#) | [Reconcilier vs flagger](#)

Drift-Long – long workflow with periodic out-of-band edits to detect.

This scenario simulates a workflow where a user occasionally edits the Revit project by hand (deleting an element, moving a wall, changing a sill height) without going through your tools. Between turns of this conversation, such out-of-band edits MAY have been applied to the active project. Your job is to surface every divergence between what your project-memory tools believe the project contains and what the live Revit document actually contains.

Run the following six-step audit in order, using only your project-memory tools (no manual inspection):

- Step 1. Establish a baseline: list how many walls, windows, and levels your tools see right now, plus the current turn / snapshot id if your tools expose one.
- Step 2. Detect drift: are there any elements present in the live Revit document but missing from your tracked state, or vice versa, or alive in both but with diverging attributes? Enumerate every divergence (element id, kind of drift, before/after). If your tools cannot detect drift by construction, say so explicitly and explain the limitation.
- Step 3. For every drift you found, decide whether to reconcile your tracked state to match Revit, or to flag the divergence for review. State your choice and the reasoning.
- Step 4. Apply this small batch of edits through your tools: raise the first wall's height by 0.1 m, lower the second wall's height by 0.1 m, persist each.
- Step 5. Detect drift again. Report.
- Step 6. Final summary: total drifts surfaced across step 2 and step 5, how many were reconciled vs flagged, and a one-line verdict ("project consistent" or "<N> drifts outstanding").

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,210	2,345
Tok out	5,001	3,829
Turns	6	13
Wall (s)	79	66

C/A tok-out 0.77 (C <= A)

Correction

A ~0% detect (honnete) / C detecte

Verdict

C moins de tokens + capacite drift

Resume (90)

Themes : Reprise save/close/reopen | Diff cross-session | Persistance historique

Resume – pick up a project after a save/close/reopen cycle.

This is a fresh Claude Code session. You have no memory of any prior turn. The Revit document open right now was edited in a previous session that is no longer running: roughly twenty new elements were added and a handful of existing elements were modified before the .rvt was saved and reopened.

Your task uses only your project-memory tools. Do not assume anything from this conversation that those tools cannot independently confirm.

Step 1 – situate yourself.

Identify the active project (id, current turn or snapshot index if your tools expose one, total node / element counts). State which tools you used to obtain this.

Step 2 – recover the prior session's diff.

Using your tools, enumerate exactly what changed between the very beginning of the project's history and the most recent persisted snapshot: which elements were created, which were modified (with old vs new values), which were deleted. If your tools cannot reach back beyond the current open document (no cross-session history by construction), say so explicitly, name the limitation, and stop here – do not invent a diff.

Step 3 – continue the work.

Add two new walls to the project, each on the highest-elevation level, 3 m long and 2.7 m high. Use the same wall type as the most recently created wall from the prior session. Persist them through your tools and confirm.

Finish with: "session resumed: <N> prior changes recovered, <M> new walls added".

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,345	2,623
Tok out	7,088	23,387
Turns	11	22
Wall (s)	110	321

C/A tok-out 3.30 (C > A)

Correction

A Step3 ok / Step2 incapable ; C les deux

Verdict

C +tokens mais SEUL a recuperer le diff (capacite)

Audit-Trail (95)

Themes : 10 cycles delete+create | Relation F2 replaced_by | Audit trail queryable

Audit-Trail – ten delete-and-replace cycles with semantic audit trail.

In the current Revit project, perform ten cycles of element replacement. Each cycle: pick one of the first ten walls in the project, delete it, then create a new wall at the same position and on the same level but with a different wall type (use any other wall type than the original type of that wall). After replacement, record in your project-memory tools the semantic relation that the new wall replaces the old one, with a one-line reason ("type swap, cycle N").

After all ten cycles:

- 1. Using your project-memory tools, enumerate the audit trail: for every "replaces" relation you recorded, give old wall id, new wall id, the recorded reason, and the timestamp / turn at which the replacement happened.
- 2. Confirm that the old walls are no longer alive in the project (they were deleted) but that the audit trail relating them to their replacements is still queryable.
- 3. If your project-memory tools have no concept of a semantic "replaces" relation between elements (i.e. you can delete and create but cannot record that one supersedes the other), state this explicitly and finish with the partial work you were able to do – do NOT invent a relation that the tools do not represent.

End with: "<N>/10 replacements done, <M>/10 with audit trail recorded".

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,210	2,605
Tok out	8,570	14,218
Turns	5	31
Wall (s)	100	158

C/A tok-out 1.66 (C > A)

Correction

A delete ok / pas d'audit ; C natif

Verdict

C +tokens mais SEUL a tracer (capacite)

P1

Themes : Create-for-real + 'what changed' | Petit modele | Overhead de base

P1 – create-for-real, then "what changed?".

You are working on a Revit building model. Do the work for REAL in the open Revit model (create the actual elements), then answer from the project state.

- 1. Create level "N0" at elevation 0 mm and level "N1" at elevation 3000 mm.
- 2. Create exactly 6 walls on level N0, 4000 mm long, 2700 mm high, laid end to end along X (wall k from x=4000*(k-1) to x=4000*k, k=1..6), all of the same basic wall type.
- 3. Then raise the height of wall #3 only, from 2700 mm to 2900 mm.

Report, with EXACT counts: how many levels and walls exist, and precisely what changed after step 3 (which wall, old height → new height) versus the state right after step 2.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,496	2,480
Tok out	12,668	6,463
Turns	22	14
Wall (s)	184	101

C/A tok-out 0.51 (C <= A)

Correction

les deux corrects

Verdict

C ~2x moins de tokens out

S4

Themes : Batch atomique + element invalide | Rollback all-or-nothing | Atomicite

S4 – atomic batch with intentional failure.

Execute this task NOW with the tools available. Do NOT ask clarifying questions – `AskUserQuestion` is not available in this mode. Do NOT defer execution or wait for confirmation.

In ONE single all-or-nothing batch, create exactly 5 new levels:

- 1. name "S4_L1", elevation 1000 mm
- 2. name "S4_L2", elevation 2000 mm
- 3. name "S4_L3", elevation 3000 mm
- 4. name "S4_BAD" – INTENTIONALLY INVALID: do NOT supply an elevation for this level. Your project-memory schema requires elevation for a Level; this element is invalid by construction. Submit it exactly as described (do NOT invent an elevation to "fix" it; the whole point is to test atomicity).
- 5. name "S4_L5", elevation 5000 mm

The batch MUST be atomic: if any element is invalid, NONE of the 5 may persist.

Apply the batch in ONE call, then report from your tools: did the batch fail (as expected), and exactly how many levels exist now? It must be unchanged from the baseline (no S4_* leaked).

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,210	2,210
Tok out	1,387	929
Turns	5	5
Wall (s)	31	23

C/A tok-out0.67 (C <= A)

Correction

les deux corrects (rollback)

Verdict

C moins de tokens (atomicite via tool)

Q

Themes : Session 5 questions | Seed 6 murs + 3 fenetres | Counts exacts

Q – multi-question session on a small model.

Execute this task NOW with the tools available. Do NOT ask clarifying questions – `AskUserQuestion` is not available in this mode. Do NOT defer execution or wait for confirmation.

Step 1 – Seed. Create level Q_N0 at elevation 0 (if not present), then create 6 walls on Q_N0, all of the same basic wall type, 2700 mm high, end-to-end along X:

- Q_WALL_1 (0,0)→(4000,0)
- Q_WALL_2 (4000,0)→(8000,0)
- Q_WALL_3 (8000,0)→(12000,0)
- Q_WALL_4 (12000,0)→(16000,0)
- Q_WALL_5 (16000,0)→(20000,0)
- Q_WALL_6 (20000,0)→(24000,0)

Then on Q_WALL_1, Q_WALL_3, Q_WALL_5 only, create exactly 1 standard single-pane window each (so 3 windows total), sill 900 / head 2100, centred at the wall midpoint.

Step 2 – Answer these 5 questions in order, one paragraph each, with exact counts from your tools (NOT from memory of what you just created):

- Q1. How many walls exist on Q_N0?
- Q2. What is the average wall height across all Q_WALL_* walls (in mm)?
- Q3. List every window with its host wall name (window_name → host).
- Q4. Which Q_WALL_* walls have NO window?
- Q5. What is the total wall length on Q_N0 (sum of all Q_WALL_* lengths) in mm?

End with a one-line summary giving Q1..Q5 as a comma-separated tuple.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,901	2,613
Tok out	31,732	10,201
Turns	35	23
Wall (s)	435	145

C/A tok-out 0.32 (C <= A)

Correction

les deux corrects

Verdict

C 3x moins de tokens (read-back KG vs Revit)

M

Themes : 10 petites editions | Persistence par edit | Read-back final

M – many small edits.

Execute this task NOW with the tools available. Do NOT ask clarifying questions – `AskUserQuestion` is not available in this mode. Do NOT defer execution or wait for confirmation.

Step 1 – Seed. Create level M_N0 at elevation 0 (if not present), then create 6 walls on M_N0, all of the same basic wall type, all 2700 mm high, end-to-end along X:

- M_WALL_1 (0,0)→(4000,0)
- M_WALL_2 (4000,0)→(8000,0)
- M_WALL_3 (8000,0)→(12000,0)
- M_WALL_4 (12000,0)→(16000,0)
- M_WALL_5 (16000,0)→(20000,0)
- M_WALL_6 (20000,0)→(24000,0)

Step 2 – Apply these 10 height edits in order, persisting each (NO restate between edits, just do them):

1. M_WALL_1 height -> 2701
2. M_WALL_2 height -> 2702
3. M_WALL_3 height -> 2703
4. M_WALL_4 height -> 2704
5. M_WALL_5 height -> 2705
6. M_WALL_6 height -> 2706
7. M_WALL_1 height -> 2710
8. M_WALL_2 height -> 2720
9. M_WALL_3 height -> 2730
10. M_WALL_4 height -> 2740

Step 3 – After all 10 edits are persisted, query your tools ONCE and report the FINAL height of each M_WALL_1..M_WALL_6 in a table. End with a one-line summary stating how many edits were applied.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,780	2,492
Tok out	21,729	6,085
Turns	32	23
Wall (s)	330	100

C/A tok-out 0.28 (C <= A)

Correction

les deux corrects

Verdict

C 3.5x moins de tokens

Impact-XL

Themes : Reachability multi-hop (cascade) | Profondeur variable (BFS) | level -> walls -> windows

Impact-XL – blast-radius of a structural deletion (multi-hop dependency).

The current Revit project has levels, walls hosted on those levels, and windows hosted on those walls. Using ONLY your project-memory tools – do NOT click through the model, do NOT inspect the .rvt by hand.

For the level named "Niveau 0", compute its full structural blast radius: everything that would be orphaned or destroyed if that level were deleted –

- every wall on that level,
- every window transitively hosted on any of those walls.

Report:

1. The number of walls on Niveau 0.
2. The number of windows transitively hosted on those walls.
3. The total blast radius (walls + windows).
4. A breakdown by hop distance from the level (hop 1 = walls, hop 2 = windows), with the count at each hop.

Do not modify anything. Walk the level -> walls -> windows dependency chain with your tools in as few calls as possible, rather than re-querying Revit element by element.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,339	2,212
Tok out	7,918	3,628
Turns	8	6
Wall (s)	113	57

C/A tok-out 0.46 (C <= A)

Correction

C 54 ok ; A ne confirme pas le host-binding

Verdict

C ~2x moins de tokens (1 sweep BFS)

TypeImpact-XL

Themes : Traversee + agregation | Dependance partagee / fan-out | type -> walls -> windows

TypeImpact-XL – shared-dependency ranking across the model.

Using ONLY your project-memory tools (do not click through the model, do not inspect the .rvt by hand):

A change to a WallType ripples to every wall of that type and onward to every window hosted on those walls. Rank the WallTypes by their "window ripple" = the total number of windows transitively hosted on walls of that type.

Report, for each WallType actually in use:

- the WallType id (and name if available),
- the number of wall instances of that type,
- the number of windows transitively hosted on those walls,

sorted by window ripple descending. End with one line: which WallType has the largest window ripple, and its size.

Do not modify anything. Use the WallType -> walls -> windows chain; make as few tool calls as possible rather than re-querying Revit per wall.

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,488	2,339
Tok out	9,769	5,200
Turns	18	9
Wall (s)	138	70

C/A tok-out 0.53 (C <= A)

Correction

les deux corrects

Verdict

C ~2x moins de tokens

Provenance

Themes : Edges semantiques F2 (replaced_by) | Traversee transitive a travers tombstones | Historique cross-suppression

Provenance – supersession chain through tombstones (semantic audit trail).

This scenario tests whether your tools can recover a chain of replacements that spans DELETED elements. Execute now; do not ask clarifying questions.

Step 1 – Build a supersession chain. Pick any one existing wall. Then, three times in a row: delete the current wall and create a replacement wall at the same position and level (use a different wall type each time), and record in your project-memory tools the semantic relation that the new wall **replaces** the previous one, with a one-line reason. After this you have a chain
original (deleted) -> r1 (deleted) -> r2 (deleted) -> r3 (alive)
with three "replaces" links.

Step 2 – Starting ONLY from the final living wall, reconstruct the full supersession history backwards: list every predecessor in order, each with its id, whether it is still alive or tombstoned, and the recorded reason for that replacement. Walk the relation transitively – do not rely on memory of what you just did in Step 1.

Step 3 – If your tools cannot traverse from a live element back through deleted predecessors (i.e. no relation persists across deletion), state this explicitly and stop – do NOT fabricate a chain.

End with: "supersession depth: <N> recovered (<alive> alive, <tombstoned> deleted)".

Resultats

	A	C
	Revit-dir	KG v2
Tok in	2,341	2,766
Tok out	4,864	11,779
Turns	10	29
Wall (s)	69	194

C/A tok-out 2.42 (C > A)

Correction

C chaine 4-deep (3 tombstoned) ; A depth 1
(incapable)

Verdict

CAPACITE exclusive du KG (A incapable)